SafeVault: finish setup today (about 45 to 60 minutes)

You already did Parts A and B: Ubuntu/WSL, the backup bucket `s3-bucket-fcc-project`, the `safevault-lite-uploader` user and its CLI profile, plus the Netlify dashboard and its public `dashboard-data.json` bucket. **No EC2, CloudWatch, SNS or CloudFront is needed for this version.**

Run every `python3 ...` command in **Ubuntu (WSL)**, not PowerShell.

Step 1: Put the new code in place (2 min)

1. Extract `SafeVault_v2.zip` in Windows. Inside is a folder called `safevault-lite`.
2. In Ubuntu, keep your old copy:

   ```
   cd ~ && mv safevault-lite safevault-lite-old && explorer.exe .
   ```

3. Copy the new `safevault-lite` folder into the File Explorer window that opens.
4. Check it:

   ```
   cd ~/safevault-lite && ls
   python3 -m unittest discover -s tests
   ```

   Expected: `OK` (53 tests). Your earlier lab data, receipts and AWS profiles live outside this folder, so they stay.

   If you never ran `python3 lite.py local-demo` before, run it once now. It creates the synthetic data and the clean hash baseline.

Step 2: Create ONE new IAM user (5 min, AWS Console)

```
python3 lite.py policies
cat ~/safevault-lite-policies/recovery-user.json
```

In **IAM -> Users -> Create user**: - Name: `safevault-lite-recovery`. No Console access. No managed policies. - **Permissions -> Add permissions -> Create inline policy -> JSON**: paste the printed JSON. Name it `SafeVaultLiteRecoveryPolicy`. - **Security credentials -> Create access key -> Command Line Interface (CLI)**.

Back in Ubuntu:

```
aws configure --profile safevault-lite-recovery      # paste the key + secret, region ap-south-1, output js
aws sts get-caller-identity --profile safevault-lite-recovery
```

The ARN must end with `user/safevault-lite-recovery`.

What this user can do: **read** backup versions and protection settings, and **write only** `dashboard-data.json`. It cannot upload, delete or unlock backups. The uploader still can't read, list or delete anything.

Step 3: Check the dashboard bucket stays public for one file (2 min)

In **S3 -> safevault-dashboard-8b4c47 -> Permissions -> Bucket policy**, there should be a statement allowing public `s3:GetObject` on `arn:aws:s3:::safevault-dashboard-8b4c47/dashboard-data.json`. Compare it with `~/safevault-lite-policies/dashboard-bucket-policy-reference.json`. If you made the file public with an **object ACL** instead, switch to this bucket policy. Each upload replaces the object, which would reset an ACL. Keep your CORS rule for the Netlify site.

Step 4: First real backup and verified recovery (5 min)

```
python3 lite.py configure-source          # skip if it says the same config already exists
python3 lite.py run-backup --select-clean
python3 lite.py verify-protection
python3 lite.py cloud-restore
```

Expected lines: - `BACKUP SUCCEEDED ... selected as clean recovery point` - `PROTECTION VERIFIED`, which shows Versioning, Object Lock, retention, encryption and public access, all read from AWS - `RECOVERY PASSED from S3: 5/5 files match the independent clean SHA-256 baseline` - `DASHBOARD UPDATED ... publicly readable.` after each command

Open https://dancing-gumdrop-8b4c47.netlify.app. You'll see the real results once you finish Step 7 (the new page), or straight away if your current page already reads `data_mode: measured`.

If `verify-protection` says Object Lock is not enabled, your bucket was created without it. Say so honestly in the presentation, or create a new bucket with Object Lock and change `BUCKET` in `lite.py`.

Step 5: Turn on automatic backups (3 min + 10 min waiting)

```
sudo apt-get update && sudo apt-get install -y cron    # only if cron is missing
sudo service cron start
python3 lite.py schedule --every 5 --install           # type SCHEDULE
```

Keep the Ubuntu window open and the laptop awake. After about 10 minutes:

```
python3 lite.py status
```

You need **at least 2 scheduled runs** marked `success`. Each run updates the dashboard by itself.

After the presentation, slow it down or stop it. Only the SafeVault line is touched:

```
python3 lite.py schedule --every 60 --install    # hourly
python3 lite.py schedule --remove                # stop
```

Step 6: Ransomware drill (2 min, rehearse once)

```
python3 lite.py ransomware-drill      # type DRILL
```

It scrambles a **disposable copy** (no malware) and tries to read, list, delete and unlock backups with the uploader key. All four must say `AccessDenied`. Then it restores the clean backup from S3 and verifies 5/5 files. Your real source folder and your cloud backups are not modified.

Step 7: Deploy the new dashboard + chatbot (10 to 15 min)

1. Get a free API key at https://console.groq.com -> API Keys.
2. In **Netlify -> your project -> Project configuration -> Environment variables -> Add a variable**: key `GROQ_API_KEY`, value = your key, scope **Functions**, mark it secret. Never put the key in `index.html` or anywhere in the code.
3. Deploy from Ubuntu. This needs Node 18 or newer. Check with `node -v`. If it's missing or old:

   ```
   curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash - && sudo apt-get install -y nodejs
   ```

   Then:

   ```
   cd ~/safevault-lite
   npx netlify-cli@latest login            # opens a browser link
   npx netlify-cli@latest link             # choose your existing site dancing-gumdrop-8b4c47
   npx netlify-cli@latest deploy --prod    # publishes website/ + the chat function (netlify.toml)
   ```

4. Open the site and click the chat button in the bottom-right corner. The header says **AI answers · Groq** once it works.

Short on time? Drag the `website` folder onto your site's **Deploys** page in Netlify. The dashboard works, and the chatbot still answers from built-in SafeVault knowledge and your live results. Only the Groq AI mode needs the CLI deploy, because drag-and-drop doesn't deploy functions.

If something fails

Message	Fix
`Wrong AWS identity`	The profile name is wrong, or `aws configure` was run without `--profile`.
`AccessDenied` on `run-backup`	Check the uploader inline policy and that the bucket allows checksum uploads. Never give it S3FullAccess.
`DASHBOARD PUBLISH FAILED`	Check the recovery user policy and the dashboard bucket name. Your backup still succeeded and was recorded.
`public read-back check did not match` / 403	Step 3, the bucket policy.
Scheduled runs not appearing	`sudo service cron start`, keep WSL open, then `cat ~/.local/state/safevault-lite/scheduled.log`.
`Retention NOT verified`	Object Lock isn't on for that version, or the clean backup is older than 7 days. Run `run-backup --select-clean` again.

Never paste access keys, the Groq key or `.aws` files into chat, screenshots or the projector.

SafeVault: 8-minute presentation plan (4 members)

One-line pitch: *Ransomware encrypts your files and then tries to delete your backups. SafeVault backs up automatically to Amazon S3, makes old clean versions undeletable, and proves it can recover them. Every result appears live on our dashboard.*

Before you start: the laptop is on, the Ubuntu window is open, the cron schedule has been running at least 10 minutes, the dashboard is open in a browser tab, and the AWS Console tab shows **only** the dashboard bucket. Never show `.aws`, access keys, the Groq key, billing or IAM key pages.

1. Member 1: Problem & architecture (1.5 min)

Problem statement: ransomware targets backups too. Walk through the flow: **cron -> Python engine -> upload-only IAM user -> private S3 bucket (Versioning + Object Lock + SSE-S3 + HTTPS-only) -> read-only recovery user -> verified restore -> sanitized JSON -> Netlify dashboard**. Why S3: very high durability, versioning, WORM lock, pay-per-use.

2. Member 2: Automation (2 min)

Run `python3 lite.py status` to show the unattended scheduled runs with their times. On the dashboard, show the **Scheduled backup runs** table and **Last successful backup**. Explain the no-overlap lock, the run log, and that failures stay visible without hiding the last success. Mention the WSL limit honestly.

3. Member 3: Protection & recovery (2 min)

Run `python3 lite.py verify-protection`. It reads Versioning, Object Lock (Governance 7 days), encryption and public-access block from AWS. Then run `python3 lite.py cloud-restore`. It downloads the **selected clean version** from S3, restores it into a new folder, and matches 5/5 SHA-256 hashes against an independent baseline. Show the protection panel.

4. Member 4: Live ransomware drill + dashboard (2.5 min)

Run `python3 lite.py ransomware-drill` (type DRILL). Narrate the 4 steps: files encrypted, 0/5 readable, the stolen key gets **AccessDenied** 4 times, then recovery of 5/5 verified. Click **Refresh from S3**: the drill panel and checklist update. Ask the chatbot "Did the drill pass?"

Close: everyone (30 s)

Limitations, in one breath: synthetic files, laptop must be on, Governance can be bypassed by an authorised admin, recovery runs on the same laptop. "Ransomware-resilient under our tested threat model, not ransomware-proof."

Likely viva questions (everyone should know these)

- **Why not just upload timestamped files?** Without Versioning and Object Lock, anyone with delete access can wipe them. Here each version is retained and the uploader cannot delete at all.
- **IAM deny vs Object Lock?** IAM stops *this identity*. Object Lock stops deletion of a retained version by *any* identity that lacks governance bypass. We show both separately.
- **Why restore an older version and not the latest?** The latest upload may already contain encrypted files. We pin the clean VersionId.
- **Governance vs Compliance?** Governance can be bypassed with a special permission (good for a lab). Compliance cannot be shortened by anyone, even root, so we didn't use it on a student account.
- **How do you know the restore is correct?** The archive SHA-256 is checked against the completion record, every file is checked against the manifest, and everything is compared with a baseline saved separately before any damage.
- **Is the dashboard live?** It refreshes every 60 seconds from S3 (polling, not streaming). Each backup or test republishes the JSON automatically.
- **Where is the Groq key?** In a Netlify environment variable, used only by a server-side function. The browser never sees it.

- **Cost?** A few small objects in S3 in Mumbai, plus free-tier Netlify and Groq. Well under a dollar for the project period.

What makes this stand out

1. It **proves** recovery instead of just claiming backups exist.
2. A **live attack drill** on the projector, with real AccessDenied results from AWS.
3. **Least privilege**: two narrow identities, no admin keys on the laptop job.
4. An **honest dashboard** that shows failures, stale data and limitations, plus an AI assistant grounded in real results.